

Project 2 — How to Run & Manual Test Cases

How to Run the Project Locally

```
# 1. Navigate to the project root
cd d:\TUHH\sem_004\HCAI\HCAI-Project

# 2. Activate the virtual environment
# PowerShell:
venv\Scripts\activate
# Git Bash:
source venv/Scripts/activate

# 3. Install required packages (if not already installed)
pip install django pandas numpy scikit-learn matplotlib palmerpenguins joblib

# 4. Run Django system check
python manage.py check

# 5. Start the development server
python manage.py runserver

# 6. Open in browser
# Home page:      http://127.0.0.1:8000/home/
# Project 1:      http://127.0.0.1:8000/project1/
# Project 2 - Home: http://127.0.0.1:8000/project2/
# Project 2 - Train: http://127.0.0.1:8000/project2/train/
# Project 2 - CF:   http://127.0.0.1:8000/project2/counterfactual/
# Project 2 - PDP/ALE: http://127.0.0.1:8000/project2/pdp-ale/

# 7. Run the automated integration test suite (optional)
python test_project2.py
```

Important: Make sure `MEDIA_URL` and `MEDIA_ROOT` are configured in `pbl/settings.py`. Plot images are saved to `media/project2_plots/`.

Note: The first time you click Train & Select Model, it trains 8 candidate models — this takes ~10–20 seconds. Subsequent pages (Counterfactuals, PDP/ALE) also retrain the model for the chosen settings.

Test Cases for Manual Testing

Use these test cases to thoroughly verify and understand the project. Each test case describes **what to do**, **what to expect**, and **what concept it demonstrates**.

Test Case 1 — Dataset Overview Page

Page: <http://127.0.0.1:8000/project2/>

Steps:

1. Open the Project 2 home page
2. Check the dataset overview card

Expected:

- Number of penguins: 333 (after dropping rows with missing values)
- Number of features: 7 (4 numerical + 3 categorical)
- Number of species: 3 (Adelie, Chinstrap, Gentoo)
- First 5 rows table is visible and scrollable
- Key concepts glossary shows definitions for all 8 terms

Concept: Understanding the data before building any model is the foundation of explainability.

Test Case 2 — Decision Tree Training with Different λ Values

Page: <http://127.0.0.1:8000/project2/train/>

Steps:

1. Select **Decision Tree**
2. Set $\lambda = 0.0 \rightarrow$ click Train & Select Model
3. Note the selected model (e.g. DT depth=unlimited) and its accuracy
4. Set $\lambda = 1.0 \rightarrow$ click Train & Select Model
5. Note the selected model (e.g. DT depth=1) and its accuracy

Expected:

- $\lambda = 0.0 \rightarrow$ selects the **most accurate** model (deeper tree, more leaves)
- $\lambda = 1.0 \rightarrow$ selects the **simplest** model (shallower tree, fewer leaves)
- The highlighted row in the candidate table changes between the two runs
- Accuracy is lower but complexity is also lower with $\lambda = 1.0$

Concept: The λ slider controls the accuracy-vs-complexity trade-off. There is no single "best" model — it depends on what you value.

Test Case 3 — Logistic Regression Training

Page: <http://127.0.0.1:8000/project2/train/>

Steps:

1. Select **Logistic Regression**
2. Set $\lambda = 0.5 \rightarrow$ click Train & Select Model
3. Scroll down to the Coefficient Table

Expected:

- Coefficient table appears with one row per encoded feature
- Green rows = features with non-zero weights (model is using them)
- Grey rows = features the model ignores (coefficients ≈ 0)
- Higher $\lambda \rightarrow$ smaller C $\rightarrow$ stronger regularisation $\rightarrow$ more zero rows

Concept: L1 regularisation creates sparse models. Non-zero coefficients tell you which features matter and how much.

Test Case 4 — λ Warning Messages

Page: <http://127.0.0.1:8000/project2/train/>

Steps:

1. Drag the λ slider to **0.95** (do not submit yet)
2. Check for the warning message below the slider
3. Drag λ slider back to **0.05**
4. Check for the info message

Expected:

- $\lambda \geq 0.8 \rightarrow$ amber warning: "Large λ strongly prefers simpler models — accuracy may drop significantly"
- $\lambda \leq 0.1 \rightarrow$ info: "Small λ prefers more accurate but possibly more complex models"
- Middle values (0.1–0.8) $\rightarrow$ no warning shown

Concept: Users should understand the implications of extreme settings before submitting.

Test Case 5 — Counterfactual: Successful Generation

Page: <http://127.0.0.1:8000/project2/counterfactual/>

Steps:

1. Select **Decision Tree**, $\lambda = 0.5$
2. Select **Row 0** from the penguin dropdown (an Adelie penguin)
3. Select target species: **Gentoo**
4. Click Generate Counterfactuals

Expected:

- Original prediction shows "Adelie" (blue badge)
- Desired species shows "Gentoo" (green badge)
- Up to 5 counterfactual rows appear in the table
- Yellow cells highlight features that changed from the original
- Distance column shows similarity (lower = more similar = better)
- Rows are sorted by distance, smallest first

Concept: Counterfactuals answer "What would need to change?" — they show the minimum realistic modifications to flip the prediction.

Test Case 6 – Counterfactual: Same-Class Warning

Page: <http://127.0.0.1:8000/project2/counterfactual/>

Steps:

1. Select **Row 0** (predicted as Adelie)
2. Select target species: **Adelie** (same as the original)
3. Click Generate Counterfactuals

Expected:

- An amber warning appears: "The model already predicts this penguin as Adelie. No changes are needed."

Concept: Counterfactuals only make sense when the target differs from the original prediction.

Test Case 7 – Counterfactual: No Results Fallback

Page: <http://127.0.0.1:8000/project2/counterfactual/>

Steps:

1. Try a combination where counterfactuals are hard to find (e.g. a Gentoo penguin with target Chinstrap at $\lambda = 0.9$)
2. If none are found, check the fallback message

Expected:

- A friendly amber message appears with 4 actionable suggestions
- No raw Python error or blank page

Concept: Graceful failure handling is important for usability.

Test Case 8 – PDP Plot

Page: <http://127.0.0.1:8000/project2/pdp-ale/>

Steps:

1. Select **Decision Tree**, $\lambda = 0.5$
2. Select feature: **flipper_length_mm**
3. Click Generate PDP & ALE

Expected:

- PDP plot shows 3 coloured lines (one per species)
- X-axis: flipper_length_mm values (range ~170–230 mm)
- Y-axis: average predicted probability (0 to 1)
- Gentoo line rises at higher flipper lengths (Gentoo have the longest flippers)
- All 3 lines sum to approximately 1.0 at every x value

Concept: PDP shows the *average* model behaviour as one feature varies.

Test Case 9 – ALE Plot

Page: <http://127.0.0.1:8000/project2/pdp-ale/>

Steps:

1. Same settings as Test Case 8 (DT, $\lambda=0.5$, flipper_length_mm)
2. Look at the ALE plot displayed below the PDP

Expected:

- ALE plot shows 3 lines with dots at bin centres
- Dashed grey horizontal line at $y = 0$ (reference)
- Values above zero = feature pushes prediction toward that species
- Values below zero = feature pushes prediction away from that species
- Shape is broadly similar to PDP but not identical

Concept: ALE only looks at local effects where real data exists. When features are correlated, ALE is more trustworthy than PDP.

Test Case 10 – PDP: Decision Tree vs Logistic Regression

Page: <http://127.0.0.1:8000/project2/pdp-ale/>

Steps:

1. Generate PDP for **Decision Tree**, feature: **bill_length_mm**
2. Then switch to **Logistic Regression**, same feature, generate again

Expected:

- DT PDP has a **step-function** shape (sharp jumps at split thresholds)
- LR PDP has **smooth S-curves** (logistic function is smooth)
- Both show the same general trend — same species favoured at same ranges

Concept: PDP reveals the model's decision logic. Different model types reason about the same data in visually different ways.

Test Case 11 — All 4 Numerical Features

Page: <http://127.0.0.1:8000/project2/pdp-a1e/>

Steps:

1. Generate PDP & ALE for each of the 4 features one by one:
 - bill_length_mm
 - bill_depth_mm
 - flipper_length_mm
 - body_mass_g

Expected:

- All 4 produce valid plots with no errors
- Each feature shows a different pattern:
 - **bill_length_mm** — Chinstrap & Gentoo favoured at high values
 - **bill_depth_mm** — Adelie & Chinstrap favoured at high values
 - **flipper_length_mm** — Gentoo strongly favoured at high values
 - **body_mass_g** — Gentoo favoured at high values

Concept: Different features influence species predictions differently. This is the core insight of feature effect plots.

Test Case 12 — Form State Persistence

Page: Any page with a form

Steps:

1. On the Train page, select **Logistic Regression** and set $\lambda = 0.73$
2. Click Train & Select Model
3. Check the form controls after results appear

Expected:

- The Logistic Regression radio button is still selected
- The lambda slider still shows 0.73
- Same behaviour on Counterfactual and PDP/ALE pages

Concept: Users should not have to re-enter settings after each submit.

Test Case 13 — Navigation

Steps:

1. Start on any Project 2 page
2. Click through all 4 nav links: Home → Train → Counterfactuals → PDP/ALE
3. Click "← Back to Project Hub"

Expected:

- All links work and load the correct page
 - The active page link is visually highlighted in the nav bar
 - Back link returns to the main home app
-

Test Case 14 — Error Suppression

Steps:

1. The Counterfactual view wraps `generate_counterfactuals()` in a try/except block
2. The PDP/ALE view wraps all plotting in a try/except block

Expected:

- If any internal error occurs, a red-bordered error card is shown with a clear, friendly message
- No raw Python traceback or Django debug page is ever shown to the user

Concept: Raw technical errors break trust. All errors should be presented in a user-friendly way.

Test Case 15 – Coefficient Table Highlight

Page: <http://127.0.0.1:8000/project2/train/>

Steps:

1. Select **Logistic Regression**, set $\lambda = 0.8$ (strong regularisation)
2. Click Train & Select Model
3. Scroll to the Coefficient Table

Expected:

- Many rows are grey (all-zero coefficients, "-" in Used? column)
- Only the most important features are green ("yes")
- Fewer non-zero coefficients than at $\lambda = 0.2$

Concept: L1 regularisation performs automatic feature selection. With strong regularisation, only the most discriminative features survive.